Community Grievance Redressal and Public Service Tracking System

Course Name & Code: UCS310 - Database Management Systems
Degree & Year: B.Tech (2nd Year)
Department: Computer Engineering

Group Members:
Diya Jain (1024031154)
Vanshika (1024030163)
Aishwarya Verma (1024030170)

Sub-Group:
2C16

Submitted To:
Dr. Simranjit Kaur

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Table of Contents

Sr. No.	Topic	Page No.
1	Introduction	1
2	Problem Statement	2
3	Objectives of the Project	3
4	Scope of the Project	4
5	Proposed System Description	5
6	Database Design	6
7	Normalization	11
8	SQL Implementation	13
9	PL/SQL Components	17
10	Transaction Management & Concurrency	22
11	Backend API Design (Flask + Python)	23
12	Sample Analytical Queries	25
13	Expected Outcomes and Results	27
14	Conclusion	28
15	Tools and Technologies Used	29

1. Introduction

Governing municipalities deal with a huge number of complaints and requests from citizens on a daily basis, from repairing infrastructure issues to sanitation problems. Good civic management demands the structured handling of citizen population data, department allocation, and resolution processes to retain trust among the populace. Yet, several municipal administrations still use outdated techniques like phone call records or application forms, which tend to create fragmented documentation and an inadequate level of transparency. The absence of a systematic process usually causes resolution delays, incomplete complaint data, and less accountability from municipal authorities.

In order to solve such inefficiencies, this project seeks to develop an efficient community grievance redressal and public services tracking system that will be driven by a database. The development of the application using the database will help to eliminate inefficiencies as far as data integrity is concerned since the application will be built on a DBMS and all data relating to the citizen's identity, the complaint, and the department to handle the grievance will be entered and therefore ensure data consistency and integrity. The application will also have the ability to track the entire lifecycle of the complaint.

The backend is implemented using a Python-based Flask framework integrated with a structured SQLite database. The architecture focuses on normalization to eliminate anomalies and improve retrieval efficiency for reporting. By enforcing constraints through the db.py layer and managing routes for authentication and complaint handling, the system aims to improve municipal accountability.

1.1 Why a Database Management System?

The choice of a relational DBMS as the backbone of the Community Grievance Redressal and Public Service Tracking System is deliberate and essential. Unlike flat-file systems or manual registers, a DBMS provides:

- **Referential Integrity through constraints** — ensuring that no complaint record can exist without being linked to a valid citizen, ward, and department.
- **Transaction Management (ACID properties)** — guaranteeing that all complaint assignments and status updates either fully complete or fully roll back to maintain a reliable audit trail.
- **Concurrent Access Control** — allowing multiple users, such as citizens, department staff, and administrators, to query or update records simultaneously without data corruption.**Centralized Data Consistency** — eliminating duplicate records and conflicting information that often occurs when grievance data is scattered across different municipal departments.

2. Problem Statement

2.1 Real-World Problem

The problems arise when the citizenry experiences issues involving the breakdown of infrastructure such as faulty roads, burnt out street lighting, and lack of water access. Currently, such problems are addressed manually through such practices as keeping record on telephone logs and emails. The main problem is the absence of a comprehensive computerized tracking system for complaints which would result in their inefficiency or even loss of records during the bureaucratic process. The inability to create an effective record keeps the officials from being aware of department responsibilities and historical data, thus damaging public trust in government.

2.2 Limitations of Existing / Manual Systems

- **No Consolidated Data System:** Since the complaints are entered manually into various physical registers, emails, and individual department files, no consolidated data system exists.
- **No Transparency:** The citizens get no complaint numbers or status updates on their complaints, so there is no transparency in the process.
- **Manual Task Distribution:** The process of assigning the particular complaint to an appropriate department is done manually by ward officers, which is a time-consuming task with chances of errors.
- **No Accountability:** There are no set deadlines for any complaint, and so it can go unaddressed forever without any accountability.

2.3 Scope of Database Usage

- **Centralized storage:** for all records involving citizens, complaints, departments, officials, assignments, and resolutions.
- **Tracking of the complaint process:** from the initial registration through to final resolution.
- **Quick access to information:** regarding complaint status and historical patterns through the use of SQL queries.
- **Ensuring data integrity:** by implementing relational mapping and database constraints.
- **Supporting decision-making:** by providing organized and reliable civic governance data.

3. Objectives of the Project

- To design a relational database for managing citizen complaints, departments, and service records using E–R modeling.
- To convert the ER design into normalized relational tables (up to 3NF) to minimize redundancy and improve efficiency.
- To structure tables to eliminate partial and transitive dependencies, ensuring reduced anomalies and improved consistency.
- To implement the database using Oracle SQL for table creation (DDL) and data manipulation (DML) including constraints.
- To apply PL/SQL features such as triggers, procedures, and functions to automate assignment, monitoring, and reporting.
- To develop stored procedures for automated complaint assignment and monitoring of deadlines.
- To implement functions to compute average resolution time and performance scores for various departments.
- To create triggers that auto-assign complaints and auto-escalate overdue cases for urgent attention.
- To create views for role-based data access, allowing citizens, officers, and administrators to view simplified, relevant datasets.

4. Scope of the Project

The proposed system is a centralised database-based grievance redressal system for handling civic complaints related to public services and infrastructure. It is designed to store, organise and track complaint information from registration to resolution in a structured relational database.

The system has three major types of users: citizens who register and track complaints, department staff or officers who update status and resolution, and an administrator who manages departments and categories and also oversees the whole system.

In this project the backend modules are complaint registration, department and category management, automatic assignment handling, status tracking and report generation using SQL queries. We use automation features like triggers and stored procedures to handle assignment logic and deadline tracking.

The project is focused on database design, normalisation, constraints and PL/SQL implementation. It does not include full frontend development, mobile applications or integration with third party government systems. This project is an academic prototype to illustrate the application of DBMS concepts in improving the grievance management.

5. Proposed System Description

The database of the proposed system is designed for a centralized municipal governance. The database includes information about citizens, wards, complaints, departments, government officials, complaint assignments, resolutions, escalations, and feedback from citizens.

In case of registering a complaint, the system receives four pieces of information: category, description, location, and priority. The system uses a unique complaint identification number to store this information. Using triggers, the system's backend logic determines the category of the complaint and assigns it to the relevant department. Based on the priority and category of the complaint, the system determines the deadline for resolution. If the complaint remains unresolved at the time of its deadline, an escalation and notification of a senior official take place.

To ensure uniformity and efficient data processing, the system uses relational databases. In addition to analytical queries that provide insight into department performance and resolution trends, SQL queries and PL/SQL functions automate the assignment and escalation processes.

5.1 System Architecture & Technology Stack

The system follows a three-tier architecture consisting of the Presentation Layer (frontend), the Logic Layer (backend API), and the Data Layer (Oracle Database).

Tier	Component	Technology / Tool	Purpose
Presentation	Frontend	HTML5, CSS3, Vanilla JavaScript	User interface for all CRUD operations and analytics
Logic	Backend API	Python 3, Flask, Flask-CORS	RESTful endpoints, business logic, Oracle DB connection pooling
Data	Database	Oracle Database (oracledb driver)	Persistent relational storage, triggers, procedures, views

6. Database Design

6.1 Entity–Relationship (ER) Diagram

6.1.1 Entities Identified

The following entities were identified during the conceptual design phase of the system:

Entity	Description	Primary Key
WARD	Stores ward-level information including councillor name and population.	ward_id
CITIZEN	Stores citizen demographic and contact information.	citizen_id
COMPLAINT	Records each complaint submitted by a citizen with category, priority, and status.	complaint_id
DEPARTMENT	Stores information about municipal departments responsible for resolving complaints.	dept_id
GOVERNMENT_OFFICIAL	Stores profiles of officials employed in departments.	official_id
ASSIGNMENT	Tracks which department and official a complaint is assigned to, with deadline.	assignment_id
RESOLUTION	Records action taken to resolve a complaint, including cost incurred.	resolution_id
ESCALATION	Logs escalation events when a complaint is not resolved within the deadline.	escalation_id
FEEDBACK	Stores citizen rating and comments after complaint resolution.	feedback_id

6.1.2 Entity–Relationship Diagram

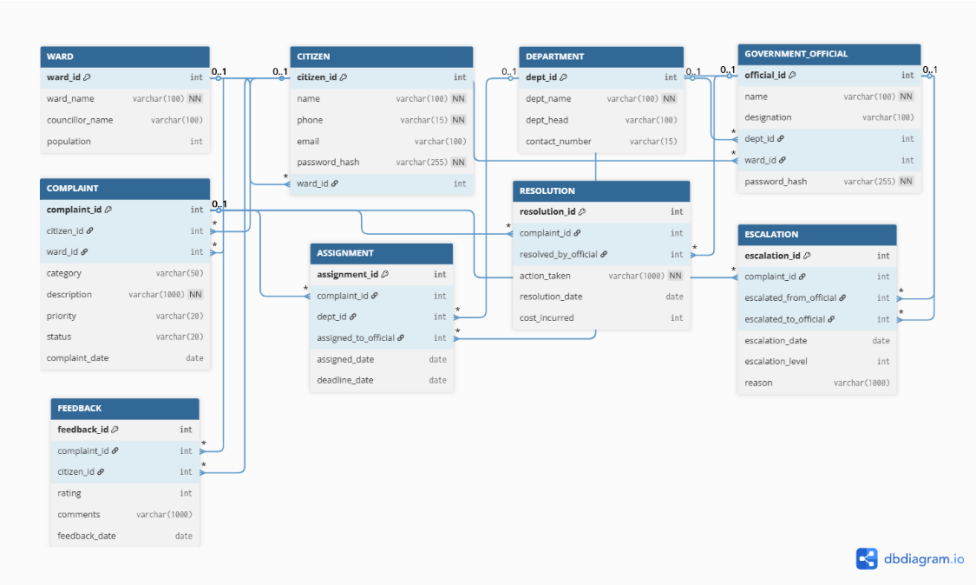

Figure 1: Entity–Relationship Diagram for NagarSeva Grievance Redressal System

6.1.3 Relationships and Cardinalities

Relationship	Entity 1	Entity 2	Cardinality	Enforced By
contains	WARD	CITIZEN	1 : N	FK ward_id in CITIZEN
registers	CITIZEN	COMPLAINT	1 : N	FK citizen_id in COMPLAINT
belongs to	WARD	COMPLAINT	1 : N	FK ward_id in COMPLAINT
assigned to	COMPLAINT	ASSIGNMENT	1 : 1	FK complaint_id in ASSIGNMENT
handles	DEPARTMENT	ASSIGNMENT	1 : N	FK dept_id in ASSIGNMENT
managed by	GOVERNMENT_OFFICIAL	ASSIGNMENT	1 : N	FK assigned_to_official in ASSIGNMENT
resolved by	COMPLAINT	RESOLUTION	1 : 1	FK complaint_id in RESOLUTION
escalated in	COMPLAINT	ESCALATION	1 : N	FK complaint_id in ESCALATION
receives	COMPLAINT	FEEDBACK	1 : 1	FK complaint_id in FEEDBACK

6.1.4 Constraints

- Each complaint should be linked to a single citizen and ward.
- Assignment records must be associated with valid complaint, department, and official entries.
- Resolution records can only be created for complaints with status ‘Resolved/Closed’
- The citizen’s email must be unique in the system.
- Feedback can be provided only once per complaint.

6.2 Relational Schema

The following section defines the complete relational schema as implemented in Oracle Database. Each table definition includes all columns, data types, constraints, and design rationale.

Table 1: WARD

Column	Data Type	Constraint	Description
ward_id	NUMBER	PK, AUTO (sequence)	Unique identifier for each ward.
ward_name	VARCHAR2(100)	NOT NULL	Name of the ward in the municipality.
councillor_name	VARCHAR2(100)	NOT NULL	Name of the elected councillor for the ward.
population	NUMBER	CHECK(population > 0)	Population of the ward. Must be a positive integer.

Table 2: CITIZEN

Column	Data Type	Constraint	Description
citizen_id	NUMBER	PK, AUTO (sequence)	Surrogate primary key uniquely identifying each citizen.
name	VARCHAR2(100)	NOT NULL	Full name of the citizen.
phone	VARCHAR2(15)	NOT NULL, UNIQUE	Mobile number; must be unique across the system.
email	VARCHAR2(100)	UNIQUE	Email address; must be unique if provided.
password_hash	VARCHAR2(256)	NOT NULL	Bcrypt-hashed password for authentication.
ward_id	NUMBER	FK → WARD(ward_id)	Links the citizen to their residential ward.

Table 3: DEPARTMENT

Column	Data Type	Constraint	Description
dept_id	NUMBER	PK, AUTO (sequence)	Unique identifier for each municipal department.
dept_name	VARCHAR2(100)	NOT NULL, UNIQUE	Name of the department (e.g., Roads, Sanitation).
dept_head	VARCHAR2(100)	NOT NULL	Name of the department head.
contact_number	VARCHAR2(15)	NOT NULL	Official contact number of the department.

Table 4: GOVERNMENT_OFFICIAL

Column	Data Type	Constraint	Description
official_id	NUMBER	PK, AUTO (sequence)	Surrogate primary key for each government official.
name	VARCHAR2(100)	NOT NULL	Full name of the official.
designation	VARCHAR2(100)	NOT NULL	Job designation (e.g., Engineer, Inspector).
dept_id	NUMBER	FK → DEPARTMENT(dept_id)	Links the official to their department.
ward_id	NUMBER	FK → WARD(ward_id)	Ward the official is responsible for.
password_hash	VARCHAR2(256)	NOT NULL	Bcrypt-hashed password for authentication.

Table 5: COMPLAINT

Column	Data Type	Constraint	Description
complaint_id	NUMBER	PK, AUTO (sequence)	Unique identifier for each registered complaint.
citizen_id	NUMBER	FK → CITIZEN(citizen_id)	Links the complaint to the submitting citizen.
ward_id	NUMBER	FK → WARD(ward_id)	Ward where the complaint originates.

Column	Data Type	Constraint	Description
category	VARCHAR2(50)	NOT NULL, CHECK(IN valid list)	Type of complaint (Roads, Water, Sanitation, Lighting, Drainage, Other).
description	CLOB	NOT NULL	Detailed free-text description of the complaint.
priority	VARCHAR2(10)	NOT NULL, CHECK(IN 'Low','Medium','High')	Priority level of the complaint.
status	VARCHAR2(20)	NOT NULL, DEFAULT 'Open'	Current status: Open, In Progress, Resolved, Closed, Escalated.
complaint_date	DATE	NOT NULL, DEFAULT SYSDATE	Date on which the complaint was registered.

Table 6: ASSIGNMENT

Column	Data Type	Constraint	Description
assignment_id	NUMBER	PK, AUTO (sequence)	Unique identifier for each assignment record.
complaint_id	NUMBER	FK → COMPLAINT(complaint_id)	The complaint being assigned.
dept_id	NUMBER	FK → DEPARTMENT(dept_id)	Department assigned to handle the complaint.
assigned_to_official	NUMBER	FK → GOVERNMENT_OFFICIAL(official_id)	Specific official assigned to the complaint.
assigned_date	DATE	NOT NULL, DEFAULT SYSDATE	Date of assignment.
deadline_date	DATE	NOT NULL	Computed deadline based on complaint priority.

Table 7: RESOLUTION

Column	Data Type	Constraint	Description
resolution_id	NUMBER	PK, AUTO (sequence)	Unique identifier for each resolution record.
complaint_id	NUMBER	FK → COMPLAINT(complaint_id), UNIQUE	The complaint that was resolved.
resolved_by_official	NUMBER	FK → GOVERNMENT_OFFICIAL(official_id)	Official who resolved the complaint.
action_taken	CLOB	NOT NULL	Description of the action taken to resolve the complaint.
resolution_date	DATE	NOT NULL, DEFAULT SYSDATE	Date on which the complaint was resolved.
cost_incurred	NUMBER(10,2)	DEFAULT 0	Cost incurred in resolving the complaint (in INR).

Table 8: ESCALATION

Column	Data Type	Constraint	Description
escalation_id	NUMBER	PK, AUTO (sequence)	Unique identifier for each escalation event.
complaint_id	NUMBER	FK → COMPLAINT(complaint_id)	The complaint being escalated.
escalated_from_official	NUMBER	FK → GOVERNMENT_OFFICIAL(official_id)	Official from whom the complaint is escalated.
escalated_to_official	NUMBER	FK → GOVERNMENT_OFFICIAL(official_id)	Senior official to whom the complaint is escalated.
escalation_level	NUMBER	NOT NULL, DEFAULT 1	Numeric level of escalation (1 = first escalation).
reason	VARCHAR2(500)	NOT NULL	Reason for escalation (e.g., deadline exceeded).
escalation_date	DATE	NOT NULL, DEFAULT SYSDATE	Date of escalation.

Table 9: FEEDBACK

Column	Data Type	Constraint	Description
feedback_id	NUMBER	PK, AUTO (sequence)	Unique identifier for each feedback record.
complaint_id	NUMBER	FK → COMPLAINT(complaint_id), UNIQUE	The complaint for which feedback is submitted.
citizen_id	NUMBER	FK → CITIZEN(citizen_id)	The citizen submitting the feedback.
rating	NUMBER(1)	NOT NULL, CHECK(rating BETWEEN 1 AND 5)	Satisfaction rating from 1 (worst) to 5 (best).
comments	VARCHAR2(1000)	NULL allowed	Optional free-text comments from the citizen.
feedback_date	DATE	NOT NULL, DEFAULT SYSDATE	Date the feedback was submitted.

7. Normalization

The entire database schema has been normalized up to Third Normal Form (3NF) to minimize redundancy, reduce update and deletion anomalies, and improve data consistency. Functional dependencies are identified for each table and used as the basis for normalization decisions.

7.1 First Normal Form (1NF)

All tables satisfy 1NF: every attribute holds atomic (indivisible) values, there are no repeating groups, and every row is uniquely identifiable by its primary key. For example, the COMPLAINT table stores one category, one priority, and one status per row — not comma-separated lists. All multi-valued data is stored in separate related tables (e.g., multiple escalations per complaint are each a separate row in the ESCALATION table).

7.2 Second Normal Form (2NF)

All non-key attributes are fully functionally dependent on the entire primary key. Since all tables in this schema use single-column surrogate primary keys (e.g., complaint_id, citizen_id, official_id), the concern of partial dependency does not arise. Every non-key attribute in every table depends entirely on its respective surrogate primary key, satisfying 2NF completely.

7.3 Third Normal Form (3NF)

There are no transitive dependencies in any table. Each non-key attribute depends only on the primary key, not on another non-key attribute. For example, in the COMPLAINT table: description, category, priority, status, and complaint_date all depend directly on complaint_id. The ward name is not stored in COMPLAINT — instead only ward_id is stored as a foreign key, avoiding the transitive dependency complaint_id → ward_id → ward_name.

Similarly, in ASSIGNMENT, the department name and official designation are not stored directly — only the foreign keys dept_id and assigned_to_official are retained, eliminating all transitive chains. The RESOLUTION table stores only the official ID (not name) as a foreign key, ensuring no transitive dependency.

7.4 Functional Dependencies Summary

Table	Functional Dependency	Normal Form Satisfied
WARD	ward_id → ward_name, councillor_name, population	3NF
CITIZEN	citizen_id → name, phone, email, password_hash, ward_id	3NF
DEPARTMENT	dept_id → dept_name, dept_head, contact_number	3NF
GOVERNMENT_OFFICIAL	official_id → name, designation, dept_id, ward_id, password_hash	3NF
COMPLAINT	complaint_id → citizen_id, ward_id, category, description, priority, status, complaint_date	3NF

Table	Functional Dependency	Normal Form Satisfied
ASSIGNMENT	assignment_id → complaint_id, dept_id, assigned_to_official, assigned_date, deadline_date	3NF
RESOLUTION	resolution_id → complaint_id, resolved_by_official, action_taken, resolution_date, cost_incurred	3NF
ESCALATION	escalation_id → complaint_id, escalated_from_official, escalated_to_official, escalation_level, reason, escalation_date	3NF
FEEDBACK	feedback_id → complaint_id, citizen_id, rating, comments, feedback_date	3NF

8. SQL Implementation

8.1 DDL: Table Creation

The following DDL statements create all nine tables with full Oracle SQL constraints, sequences for auto-increment, and referential integrity rules. The implementation uses Oracle's CREATE SEQUENCE and triggers for auto-increment functionality.

8.1.1 WARD Table DDL

```
CREATE SEQUENCE ward_seq START WITH 1 INCREMENT BY 1;

CREATE TABLE WARD (
    ward_id        NUMBER        PRIMARY KEY,
    ward_name      VARCHAR2(100) NOT NULL,
    councillor_name VARCHAR2(100) NOT NULL,
    population     NUMBER        CHECK(population > 0)
);

CREATE OR REPLACE TRIGGER ward_bir
BEFORE INSERT ON WARD FOR EACH ROW
BEGIN
    SELECT ward_seq.NEXTVAL INTO :NEW.ward_id FROM DUAL;
END;
```

8.1.2 CITIZEN Table DDL

```
CREATE SEQUENCE citizen_seq START WITH 1 INCREMENT BY 1;

CREATE TABLE CITIZEN (
    citizen_id    NUMBER         PRIMARY KEY,
    name          VARCHAR2(100)  NOT NULL,
    phone         VARCHAR2(15)   NOT NULL UNIQUE,
    email         VARCHAR2(100)  UNIQUE,
    password_hash VARCHAR2(256)  NOT NULL,
    ward_id       NUMBER         REFERENCES WARD(ward_id)
);

CREATE OR REPLACE TRIGGER citizen_bir
BEFORE INSERT ON CITIZEN FOR EACH ROW
BEGIN
    SELECT citizen_seq.NEXTVAL INTO :NEW.citizen_id FROM DUAL;
END;
```

8.1.3 COMPLAINT Table DDL

```
CREATE SEQUENCE complaint_seq START WITH 1 INCREMENT BY 1;

CREATE TABLE COMPLAINT (
    complaint_id   NUMBER        PRIMARY KEY,
    citizen_id     NUMBER        NOT NULL REFERENCES CITIZEN(citizen_id),
    ward_id        NUMBER        NOT NULL REFERENCES WARD(ward_id),
    category       VARCHAR2(50)  NOT NULL
                   CHECK(category IN ('Roads','Water Supply','Sanitation',
                                      'Street Lighting','Drainage','Other')),
    description    CLOB          NOT NULL,
```

```
    priority       VARCHAR2(10)  NOT NULL
                   CHECK(priority IN ('Low','Medium','High')),
    status         VARCHAR2(20)  NOT NULL DEFAULT 'Open'
                   CHECK(status IN ('Open','In
Progress','Resolved','Closed','Escalated')),
    complaint_date DATE          NOT NULL DEFAULT SYSDATE
);

CREATE OR REPLACE TRIGGER complaint_bir
BEFORE INSERT ON COMPLAINT FOR EACH ROW
BEGIN
    SELECT complaint_seq.NEXTVAL INTO :NEW.complaint_id FROM DUAL;
    IF :NEW.complaint_date IS NULL THEN
        :NEW.complaint_date := SYSDATE;
    END IF;
END;
```

8.1.4 ASSIGNMENT Table DDL

```
CREATE SEQUENCE assignment_seq START WITH 1 INCREMENT BY 1;

CREATE TABLE ASSIGNMENT (
    assignment_id        NUMBER   PRIMARY KEY,
    complaint_id         NUMBER   NOT NULL REFERENCES
COMPLAINT(complaint_id),
    dept_id              NUMBER   NOT NULL REFERENCES DEPARTMENT(dept_id),
    assigned_to_official NUMBER   NOT NULL
                         REFERENCES GOVERNMENT_OFFICIAL(official_id),
    assigned_date        DATE     NOT NULL DEFAULT SYSDATE,
    deadline_date        DATE     NOT NULL
);
```

8.1.5 RESOLUTION, ESCALATION & FEEDBACK Tables DDL

```
-- RESOLUTION
CREATE TABLE RESOLUTION (
    resolution_id      NUMBER         PRIMARY KEY,
    complaint_id       NUMBER         NOT NULL UNIQUE
                       REFERENCES COMPLAINT(complaint_id),
    resolved_by_official NUMBER       NOT NULL
                       REFERENCES GOVERNMENT_OFFICIAL(official_id),
    action_taken       CLOB           NOT NULL,
    resolution_date    DATE           NOT NULL DEFAULT SYSDATE,
    cost_incurred      NUMBER(10,2)   DEFAULT 0
);

-- ESCALATION
CREATE TABLE ESCALATION (
    escalation_id          NUMBER        PRIMARY KEY,
    complaint_id           NUMBER        NOT NULL
                           REFERENCES COMPLAINT(complaint_id),
    escalated_from_official NUMBER       NOT NULL
                           REFERENCES GOVERNMENT_OFFICIAL(official_id),
    escalated_to_official  NUMBER        NOT NULL
                           REFERENCES GOVERNMENT_OFFICIAL(official_id),
    escalation_level       NUMBER        NOT NULL DEFAULT 1,
    reason                 VARCHAR2(500) NOT NULL,
```

```
    escalation_date        DATE          NOT NULL DEFAULT SYSDATE
);

-- FEEDBACK
CREATE TABLE FEEDBACK (
    feedback_id   NUMBER          PRIMARY KEY,
    complaint_id  NUMBER          NOT NULL UNIQUE
                  REFERENCES COMPLAINT(complaint_id),
    citizen_id    NUMBER          NOT NULL
                  REFERENCES CITIZEN(citizen_id),
    rating        NUMBER(1)       NOT NULL CHECK(rating BETWEEN 1 AND 5),
    comments      VARCHAR2(1000),
    feedback_date DATE            NOT NULL DEFAULT SYSDATE
);
```

8.2 DML: Data Manipulation

The following are representative DML statements used to insert, update, and retrieve data in the system.

8.2.1 Sample INSERT Statements

```
-- Insert a ward
INSERT INTO WARD (ward_name, councillor_name, population)
VALUES ('Sector 7', 'Ramesh Kumar', 45000);

-- Insert a citizen
INSERT INTO CITIZEN (name, phone, email, password_hash, ward_id)
VALUES ('Anita Sharma', '9876543210',
        'anita@email.com', 'hashed_password_here', 1);

-- Insert a complaint
INSERT INTO COMPLAINT (citizen_id, ward_id, category,
                       description, priority)
VALUES (1, 1, 'Roads',
        'Large pothole near bus stop causing accidents.',
        'High');
```

8.2.2 UPDATE Statements

```
-- Update complaint status
UPDATE COMPLAINT SET status = 'In Progress'
WHERE complaint_id = 1;

-- Update resolution cost
UPDATE RESOLUTION SET cost_incurred = 15000
WHERE complaint_id = 1;
```

8.3 Views Created

Two database views are created to provide simplified, joined datasets for application-layer consumption. These views avoid repeated joins in application code and ensure role-based data exposure.

8.3.1 View: admin_dashboard_view

This view joins COMPLAINT, CITIZEN, WARD, ASSIGNMENT, DEPARTMENT, and GOVERNMENT_OFFICIAL to provide a comprehensive snapshot of every complaint for the admin dashboard. The backend API queries this view for the main complaint listing endpoint.

```
CREATE OR REPLACE VIEW admin_dashboard_view AS
SELECT
    c.complaint_id,
    c.category,
    c.description,
    c.priority,
    c.status,
    c.complaint_date,
    ci.name        AS citizen_name,
    ci.phone       AS citizen_phone,
    w.ward_name,
    w.ward_id,
    d.dept_name,
    d.dept_id,
    o.name         AS official_name,
    a.assigned_date,
    a.deadline_date
FROM COMPLAINT c
JOIN CITIZEN          ci ON ci.citizen_id   = c.citizen_id
JOIN WARD              w ON w.ward_id        = c.ward_id
LEFT JOIN ASSIGNMENT   a ON a.complaint_id   = c.complaint_id
LEFT JOIN DEPARTMENT   d ON d.dept_id        = a.dept_id
LEFT JOIN GOVERNMENT_OFFICIAL o ON o.official_id = a.assigned_to_official;
```

8.3.2 View: citizen_complaint_view

This view provides a citizen-facing snapshot of complaints and their current resolution status, used by the citizen portal to show complaint history.

```
CREATE OR REPLACE VIEW citizen_complaint_view AS
SELECT
    c.complaint_id,
    c.category,
    c.description,
    c.priority,
    c.status,
    c.complaint_date,
    ci.citizen_id,
    ci.name   AS citizen_name,
    w.ward_name,
    d.dept_name,
    a.deadline_date,
    r.action_taken,
    r.resolution_date,
    r.cost_incurred
FROM COMPLAINT c
JOIN CITIZEN         ci ON ci.citizen_id   = c.citizen_id
JOIN WARD             w ON w.ward_id        = c.ward_id
LEFT JOIN ASSIGNMENT  a ON a.complaint_id   = c.complaint_id
LEFT JOIN DEPARTMENT  d ON d.dept_id        = a.dept_id
LEFT JOIN RESOLUTION  r ON r.complaint_id   = c.complaint_id;
```

9. PL/SQL Components

9.1 Stored Procedures

9.1.1 Procedure: register_complaint

This stored procedure encapsulates the entire complaint registration workflow inside a single atomic transaction. It accepts citizen ID, ward ID, category, description, and priority as inputs, inserts the complaint record, and then automatically assigns it to the correct department based on category mapping. It computes the deadline date based on priority: High = 3 days, Medium = 7 days, Low = 14 days. The output parameter returns the generated complaint_id to the calling application.

```
CREATE OR REPLACE PROCEDURE register_complaint(
    p_citizen_id  IN  NUMBER,
    p_ward_id     IN  NUMBER,
    p_category    IN  VARCHAR2,
    p_description IN  VARCHAR2,
    p_priority    IN  VARCHAR2,
    p_complaint_id OUT NUMBER
) AS
    v_dept_id    NUMBER;
    v_official_id NUMBER;
    v_deadline   DATE;
BEGIN
    -- Insert complaint
    INSERT INTO COMPLAINT
        (citizen_id, ward_id, category, description, priority)
    VALUES
        (p_citizen_id, p_ward_id, p_category, p_description, p_priority)
    RETURNING complaint_id INTO p_complaint_id;

    -- Determine deadline by priority
    v_deadline := CASE p_priority
                    WHEN 'High'   THEN SYSDATE + 3
                    WHEN 'Medium' THEN SYSDATE + 7
                    ELSE SYSDATE + 14
                  END;

    -- Find matching department for category
    SELECT dept_id INTO v_dept_id
    FROM DEPARTMENT
    WHERE UPPER(dept_name) LIKE '%' || UPPER(
        CASE p_category
            WHEN 'Roads'          THEN 'Road'
            WHEN 'Water Supply'   THEN 'Water'
            WHEN 'Sanitation'     THEN 'Sanit'
            WHEN 'Street Lighting'THEN 'Light'
            WHEN 'Drainage'       THEN 'Drain'
            ELSE 'General'
        END) || '%'
    AND ROWNUM = 1;

    -- Pick an available official from that department
    SELECT official_id INTO v_official_id
    FROM GOVERNMENT_OFFICIAL
    WHERE dept_id = v_dept_id AND ROWNUM = 1;
```

```
    -- Create assignment
    INSERT INTO ASSIGNMENT
        (complaint_id, dept_id, assigned_to_official, deadline_date)
    VALUES
        (p_complaint_id, v_dept_id, v_official_id, v_deadline);

    COMMIT;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        ROLLBACK;
        RAISE_APPLICATION_ERROR(-20001,
            'No department/official found for category: ' || p_category);
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END register_complaint;
```

9.1.2 Procedure: resolve_complaint

This stored procedure handles complaint resolution. It inserts a resolution record with the action taken and cost incurred, then updates the complaint status to 'Resolved'. Exception handling ensures the operation is rolled back atomically if any error occurs.

```
CREATE OR REPLACE PROCEDURE resolve_complaint(
    p_complaint_id   IN NUMBER,
    p_official_id    IN NUMBER,
    p_action_taken   IN VARCHAR2,
    p_cost_incurred  IN NUMBER
) AS
BEGIN
    INSERT INTO RESOLUTION
        (complaint_id, resolved_by_official, action_taken,
         resolution_date, cost_incurred)
    VALUES
        (p_complaint_id, p_official_id, p_action_taken,
         SYSDATE, p_cost_incurred);

    UPDATE COMPLAINT
       SET status = 'Resolved'
     WHERE complaint_id = p_complaint_id;

    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END resolve_complaint;
```

9.2 Stored Functions

9.2.1 Function: get_avg_resolution_time

This deterministic stored function calculates the average time (in days) taken to resolve complaints for a given department. It queries the RESOLUTION and ASSIGNMENT tables and returns a numeric value representing the average resolution duration. This function is called by the analytics endpoint for department performance metrics.

```
CREATE OR REPLACE FUNCTION get_avg_resolution_time(
    p_dept_id IN NUMBER
) RETURN NUMBER IS
    v_avg_days NUMBER;
BEGIN
    SELECT AVG(r.resolution_date - a.assigned_date)
    INTO   v_avg_days
    FROM   RESOLUTION      r
    JOIN   COMPLAINT        c ON c.complaint_id  = r.complaint_id
    JOIN   ASSIGNMENT       a ON a.complaint_id  = c.complaint_id
    WHERE  a.dept_id = p_dept_id;

    RETURN NVL(ROUND(v_avg_days, 2), 0);
EXCEPTION
    WHEN OTHERS THEN RETURN 0;
END get_avg_resolution_time;
```

9.2.2 Function: get_dept_performance_score

This function computes a quantitative performance score for a department. It calculates the percentage of complaints resolved on time (before the deadline). A score of 100 indicates all complaints were resolved within the deadline, while 0 indicates none were. The score is returned as a NUMBER between 0 and 100.

```
CREATE OR REPLACE FUNCTION get_dept_performance_score(
    p_dept_id IN NUMBER
) RETURN NUMBER IS
    v_total    NUMBER;
    v_on_time  NUMBER;
    v_score    NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO   v_total
    FROM   ASSIGNMENT a
    WHERE  a.dept_id = p_dept_id;

    SELECT COUNT(*)
    INTO   v_on_time
    FROM   ASSIGNMENT  a
    JOIN   RESOLUTION  r ON r.complaint_id = a.complaint_id
    WHERE  a.dept_id = p_dept_id
      AND  r.resolution_date <= a.deadline_date;

    IF v_total = 0 THEN
        RETURN 0;
    END IF;
```

```
    v_score := ROUND((v_on_time / v_total) * 100, 2);
    RETURN v_score;
EXCEPTION
    WHEN OTHERS THEN RETURN 0;
END get_dept_performance_score;
```

9.3 Triggers

9.3.1 Trigger: trg_auto_escalate

This trigger fires AFTER an INSERT on the ASSIGNMENT table. It checks if the associated complaint is already past its deadline (to handle backdated assignments or immediate escalation scenarios). If the deadline is already exceeded, it creates an ESCALATION record automatically and updates the complaint status to 'Escalated'. This ensures no overdue complaint slips through without management visibility.

```
CREATE OR REPLACE TRIGGER trg_auto_escalate
AFTER INSERT ON ASSIGNMENT
FOR EACH ROW
DECLARE
    v_senior_id NUMBER;
BEGIN
    -- Check if already overdue at time of assignment
    IF :NEW.deadline_date < SYSDATE THEN
        -- Find a senior official in same department
        SELECT official_id INTO v_senior_id
        FROM   GOVERNMENT_OFFICIAL
        WHERE  dept_id = :NEW.dept_id
          AND  official_id <> :NEW.assigned_to_official
          AND  ROWNUM = 1;

        INSERT INTO ESCALATION (
            complaint_id, escalated_from_official,
            escalated_to_official, escalation_level, reason
        ) VALUES (
            :NEW.complaint_id, :NEW.assigned_to_official,
            v_senior_id, 1,
            'Auto-escalated: Deadline exceeded at time of assignment'
        );

        UPDATE COMPLAINT
           SET status = 'Escalated'
         WHERE complaint_id = :NEW.complaint_id;
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN NULL; -- No senior official available
    WHEN OTHERS THEN NULL;
END trg_auto_escalate;
```

9.3.2 Trigger: trg_complaint_status_update

This BEFORE UPDATE trigger on the COMPLAINT table validates status transitions. It ensures that a complaint can only move forward in its lifecycle: from Open to In Progress, from In Progress to Resolved or Escalated, and from Resolved to Closed. If an invalid status transition is attempted (e.g., directly from Open to Closed), the trigger raises an application error and the update is rejected.

```
CREATE OR REPLACE TRIGGER trg_complaint_status_update
BEFORE UPDATE OF status ON COMPLAINT
FOR EACH ROW
BEGIN
    -- Validate status transition
    IF :OLD.status = 'Open'
       AND :NEW.status NOT IN ('In Progress', 'Escalated') THEN
        RAISE_APPLICATION_ERROR(-20002,
            'Invalid status transition from Open to ' || :NEW.status);
    ELSIF :OLD.status = 'In Progress'
       AND :NEW.status NOT IN ('Resolved', 'Escalated') THEN
        RAISE_APPLICATION_ERROR(-20003,
            'Invalid status transition from In Progress to ' || :NEW.status);
    ELSIF :OLD.status = 'Resolved'
       AND :NEW.status <> 'Closed' THEN
        RAISE_APPLICATION_ERROR(-20004,
            'Resolved complaint can only be Closed.');
    END IF;
END trg_complaint_status_update;
```

9.4 Exception Handling

The exception handling pattern used in all the stored procedures/functions follows the ORACLE approach for structuring exceptions through the EXCEPTION clause along with the WHEN OTHERS THEN clause and ROLLBACK statement. For instance, the register_complaint procedure employs the WHEN NO_DATA_FOUND clause in order to find out whether there is any entry for the department and its corresponding official; otherwise, an application error is raised. In the resolve_complaint procedure, the transaction is rolled back completely in case either of the operations INSERT into RESOLUTION or UPDATE on the COMPLAINT fails.

This mechanism guarantees that only those transactions succeed whose parts like registering a complaint without assigning it are completed in totality. Any failure at any point is reported as an error by the Flask application server to the front-end application.

10. Transaction Management & Concurrency

10.1 ACID Compliance

ACID Property	Implementation in This Project
Atomicity	The register_complaint and resolve_complaint stored procedures wrap all DML operations in a single logical transaction using COMMIT at the end and ROLLBACK in exception handlers. Either all inserts succeed together, or none persist.
Consistency	CHECK constraints on category, priority, status, and rating; NOT NULL constraints; and foreign key constraints ensure the database transitions only to valid states after every operation.
Isolation	Oracle's default READ COMMITTED isolation level ensures that concurrent complaint submissions do not read each other's uncommitted changes. The connection pool (oracledb.SessionPool) manages concurrent sessions safely.
Durability	Oracle Database writes committed transactions to the redo log before acknowledging success, ensuring durability across server crashes and restarts.

10.2 Referential Integrity Rules

Foreign Key	ON DELETE Rule	Rationale
CITIZEN.ward_id → WARD.ward_id	RESTRICT (default)	Prevents deletion of a ward that has registered citizens.
COMPLAINT.citizen_id → CITIZEN.citizen_id	RESTRICT (default)	Prevents citizen deletion if they have submitted complaints.
COMPLAINT.ward_id → WARD.ward_id	RESTRICT (default)	Prevents ward deletion if it has associated complaints.
ASSIGNMENT.complaint_id → COMPLAINT.complaint_id	RESTRICT (default)	Assignment records must reference a valid complaint.
ASSIGNMENT.dept_id → DEPARTMENT.dept_id	RESTRICT (default)	Prevents department deletion if it has active assignments.
RESOLUTION.complaint_id → COMPLAINT.complaint_id	RESTRICT (default)	Resolution records are tied to existing complaints.
ESCALATION.complaint_id → COMPLAINT.complaint_id	RESTRICT (default)	Escalation records must reference a valid complaint.
FEEDBACK.complaint_id → COMPLAINT.complaint_id	RESTRICT (default)	Feedback is tied to an existing complaint.

11. Backend API Design (Flask + Python)

The backend exposes a RESTful API under the /api/ prefix. All endpoints interact directly with the Oracle Database via the oracledb.SessionPool connection pool. The Flask application is organized using Blueprint modules, each responsible for a specific entity domain.

Method	Endpoint	Description	DB Object Used
POST	/api/register	Register a new citizen account	INSERT into CITIZEN with RETURNING
POST	/api/login	Login for citizen, official, or admin	SELECT from CITIZEN / GOVERNMENT_OFFICIAL
POST	/api/logout	Logout (session invalidation)	Application-layer only
GET	/api/complaints	List all complaints (with filters)	SELECT from admin_dashboard_view
POST	/api/complaints	Submit a new complaint	CALL register_complaint() procedure
GET	/api/complaints/:id	Get a single complaint by ID	SELECT from admin_dashboard_view
GET	/api/complaints/citizen/:id	Get all complaints for a citizen	SELECT from citizen_complaint_view
GET	/api/complaints/official/:id	Get complaints assigned to an official	JOIN COMPLAINT, ASSIGNMENT, WARD, GOVERNMENT_OFFICIAL
PUT	/api/complaints/:id/status	Update complaint status	UPDATE COMPLAINT status
POST	/api/resolutions	Resolve a complaint	CALL resolve_complaint() procedure
POST	/api/feedback	Submit citizen feedback	INSERT into FEEDBACK with RETURNING
GET	/api/feedback/:complaint_id	Get feedback for a complaint	SELECT from FEEDBACK
GET	/api/departments	List all departments	SELECT from DEPARTMENT
GET	/api/departments/:id/stats	Get dept performance stats	CALL get_avg_resolution_time() & get_dept_performance_score()
GET	/api/wards	List all wards	SELECT from WARD
GET	/api/analytics/complaints-by-department	Complaint count per department	JOIN DEPARTMENT, ASSIGNMENT
GET	/api/analytics/complaints-by-status	Complaint count by status	GROUP BY on COMPLAINT
GET	/api/analytics/complaints-by-month	Monthly complaint trend (6 months)	TO_CHAR, ADD_MONTHS on COMPLAINT
GET	/api/analytics/top-wards	Top 5 wards by complaint count	JOIN WARD, COMPLAINT
GET	/api/analytics/overdue	List overdue complaints	JOIN COMPLAINT, ASSIGNMENT, GOVERNMENT_OFFICIAL, DEPARTMENT

Method	Endpoint	Description	DB Object Used
GET	/api/analytics/escalated	List escalated complaints	JOIN ESCALATION, COMPLAINT

12. Sample Analytical Queries

The following SQL queries demonstrate the analytical capabilities of the NagarSeva system. These queries are used in the analytics dashboard to support data-driven governance decisions.

Query 1: Complaints Assigned to Each Department

```
SELECT d.dept_name, COUNT(a.complaint_id) AS total_complaints
FROM DEPARTMENT d
LEFT JOIN ASSIGNMENT a ON d.dept_id = a.dept_id
GROUP BY d.dept_name
ORDER BY total_complaints DESC;
```

Query 2: Overdue Complaints with Official and Department Details

```
SELECT c.complaint_id, c.category, a.deadline_date,
       o.name AS official_name, d.dept_name
FROM   COMPLAINT c
JOIN   ASSIGNMENT          a ON a.complaint_id  = c.complaint_id
JOIN   GOVERNMENT_OFFICIAL o ON o.official_id   = a.assigned_to_official
JOIN   DEPARTMENT          d ON d.dept_id        = a.dept_id
WHERE  a.deadline_date < SYSDATE
  AND  c.status NOT IN ('Resolved','Closed')
ORDER BY a.deadline_date ASC;
```

Query 3: Monthly Complaint Trend (Last 6 Months)

```
SELECT TO_CHAR(complaint_date, 'YYYY-MM') AS month,
       COUNT(*) AS complaints_filed
FROM   COMPLAINT
WHERE  complaint_date >= ADD_MONTHS(SYSDATE, -6)
GROUP  BY TO_CHAR(complaint_date, 'YYYY-MM')
ORDER  BY month ASC;
```

Query 4: Top 5 Wards by Complaint Volume

```
SELECT w.ward_name, COUNT(c.complaint_id) AS complaint_count
FROM   WARD w
LEFT JOIN COMPLAINT c ON c.ward_id = w.ward_id
GROUP  BY w.ward_name
ORDER  BY complaint_count DESC
FETCH  FIRST 5 ROWS ONLY;
```

Query 5: Department Performance Score and Average Resolution Time

```
SELECT d.dept_name,
       get_avg_resolution_time(d.dept_id)    AS avg_days_to_resolve,
       get_dept_performance_score(d.dept_id) AS performance_score
FROM   DEPARTMENT d
ORDER  BY performance_score DESC;
```

Query 6: Escalated Complaints with Level and Reason

```
SELECT c.complaint_id, c.category,
       e.escalation_level, e.reason, e.escalation_date
FROM   ESCALATION e
JOIN   COMPLAINT   c ON c.complaint_id = e.complaint_id
ORDER  BY e.escalation_date DESC;
```

Query 7: Average Citizen Satisfaction Rating per Department

```
SELECT d.dept_name,
       ROUND(AVG(f.rating), 2) AS avg_satisfaction,
       COUNT(f.feedback_id)   AS feedback_count
FROM   FEEDBACK f
JOIN   COMPLAINT   c ON c.complaint_id = f.complaint_id
JOIN   ASSIGNMENT  a ON a.complaint_id  = c.complaint_id
JOIN   DEPARTMENT  d ON d.dept_id       = a.dept_id
GROUP  BY d.dept_name
ORDER  BY avg_satisfaction DESC;
```

13. Expected Outcomes and Results

Outcome	Status	Details
Fully normalized database schema	Achieved	All 9 tables are in 3NF with no partial or transitive dependencies. Foreign keys enforce referential integrity across the entire schema.
Automated complaint assignment	Achieved	register_complaint procedure maps complaint category to department and assigns an official automatically with computed deadline.
Automated escalation detection	Achieved	trg_auto_escalate trigger fires on ASSIGNMENT INSERT and creates ESCALATION records for overdue complaints automatically.
Status lifecycle validation	Achieved	trg_complaint_status_update enforces valid status transitions, preventing invalid state changes via any client.
Quantitative performance scoring	Achieved	get_dept_performance_score() returns on-time resolution percentage per department for the analytics dashboard.
Average resolution time tracking	Achieved	get_avg_resolution_time() computes mean days to resolve per department, enabling workload benchmarking.
Transactional complaint resolution	Achieved	resolve_complaint procedure ensures atomicity of resolution INSERT and status UPDATE with ROLLBACK on failure.
Analytical views	Achieved	admin_dashboard_view and citizen_complaint_view expose denormalized data for fast API consumption.
RESTful API for all CRUD operations	Achieved	21 API endpoints cover all entities — complaints, resolutions, feedback, departments, wards, analytics, and auth.
Role-based data access	Achieved	Three user roles (citizen, official, admin) with separate API endpoints and views exposing role-appropriate data.

14. Conclusion

The Community Grievance Redressal and Public Service Tracking System (NagarSeva) demonstrates how a well-designed relational database can be the backbone of a transparency-critical civic governance application. The main contribution of the project is not the user facing application but the database layer, a normalised schema with automated trigger logic, transactional stored procedures, quantitative performance functions and efficient analytical views.

An interesting new method of assigning compla ints and escalating them through the use of "triggers" provides assurance that no record will be created if there is no department assigned and no calculation for when the complaint must be resolved. If a complaint is entered into the database using any application or API client, then the database guarantees that this action has occurred. Consequ

ently, there will be no exceptions to this principle so that the civic accountability of public service will be enforced by the database at all times.

The performance scoring function provides administrators with a single number to use to compare the departments against each other. This gives the administrator a means to identify the departments that are "under the bar" and gives them a way to prioritize what departments require intervention. The company's database is also able to provide administrators with an analytical view of the data as well as the ability to use complex SQL queries to generate the necessary information to improve the quality and quantity of public service delivery.

The importance of the concepts of a DBMS (e.g., normalization, constraints, triggers, stored procedures, functions, views, transaction control) to build the type of system required to assure the accuracy of the data and thereby have a means of sustaining public trust in the governance process cannot be understated. This project provides evidence of how the project supports Sustainable Development Goal 16 (Peace, Justice, and Strong Institutions) to enhance civic management through improved transparency and accountability.

15. Tools and Technologies Used

Category	Tool / Technology	Version / Notes
DBMS	Oracle Database	Oracle Database 21c; used with oracledb Python driver
PL/SQL	Oracle SQL / PL/SQL	DDL, DML, stored procedures, functions, triggers, views
DB Interface Tool	Oracle Live SQL	For query testing, schema visualization, and procedure execution
Backend Runtime	Python 3.14	Application server and Oracle DB connection pool management
Backend Framework	Flask + Flask-CORS	RESTful API routing under /api/ with Blueprint modularization
DB Driver	oracledb (python-oracledb)	Oracle's official Python driver with SessionPool support
Auth Library	Werkzeug (werkzeug.security)	Bcrypt-based password hashing for citizen and official authentication
Frontend	HTML5, CSS3, Vanilla JavaScript	No framework dependency; modular page-based structure
Development Tool	VS Code	Primary IDE for Python, JavaScript, and SQL development
Version Control	Git	Source code versioning throughout the project